

Diseño y Programación Orientados a Objetos (ELO329)

Documentación Técnica

Tarea 2: Simulador Gráfico de ELOTelTags y Aplicación Find My

Integrantes:

Martin Layseca

Diego Villanueva

Joaquin Torres

Eduardo Canales

17 de mayo de 2026

1. Descripción Arquitectural

Esta aplicación implementa una simulación guiada por eventos utilizando la biblioteca gráfica **JavaFX** para modelar el rastro, proximidad y reporte de dispositivos móviles en un plano coordinado. La arquitectura interna se fundamenta estrictamente en el patrón de diseño **Modelo-Vista-Controlador (MVC)**, garantizando una separación limpia de responsabilidades dentro del sistema:

- **Modelo:** Clases que heredan de la superclase abstracta **Equipo** (**Celular**, **EloTelTag**, **Tablet**). Estas clases encapsulan los datos lógicos del negocio, controlando de forma interna las coordenadas matemáticas de posición (x, y) , la rapidez física y los ángulos vectoriales de desplazamiento dinámico.
- **Vista:** Clases que extienden de **EquipoView** (**CellularView**, **EloTelTagView**, **TabletView**). Son las encargadas de la representación en la interfaz de usuario, gestionando el renderizado de los nodos geométricos en la escena, la animación de los pulsos expansivos de los radares y la instanciación visual de menús de interacción.
- **Controlador:** La clase **Territory** actúa como el motor central del sistema. Controla el bucle de tiempo principal (**Timeline**), ejecuta físicamente el desplazamiento de los equipos asegurando el rebote perimetral en los bordes del mapa y gatilla las rutinas periódicas de colisiones euclidianas para centralizar las tramas de datos hacia la nube.

2. Diagrama de Clases UML

El siguiente diagrama detalla las relaciones de herencia, composición y agregación que dan estructura a la aplicación en su etapa final, reflejando el desacoplamiento de la lógica gráfica y el flujo de control centralizado:

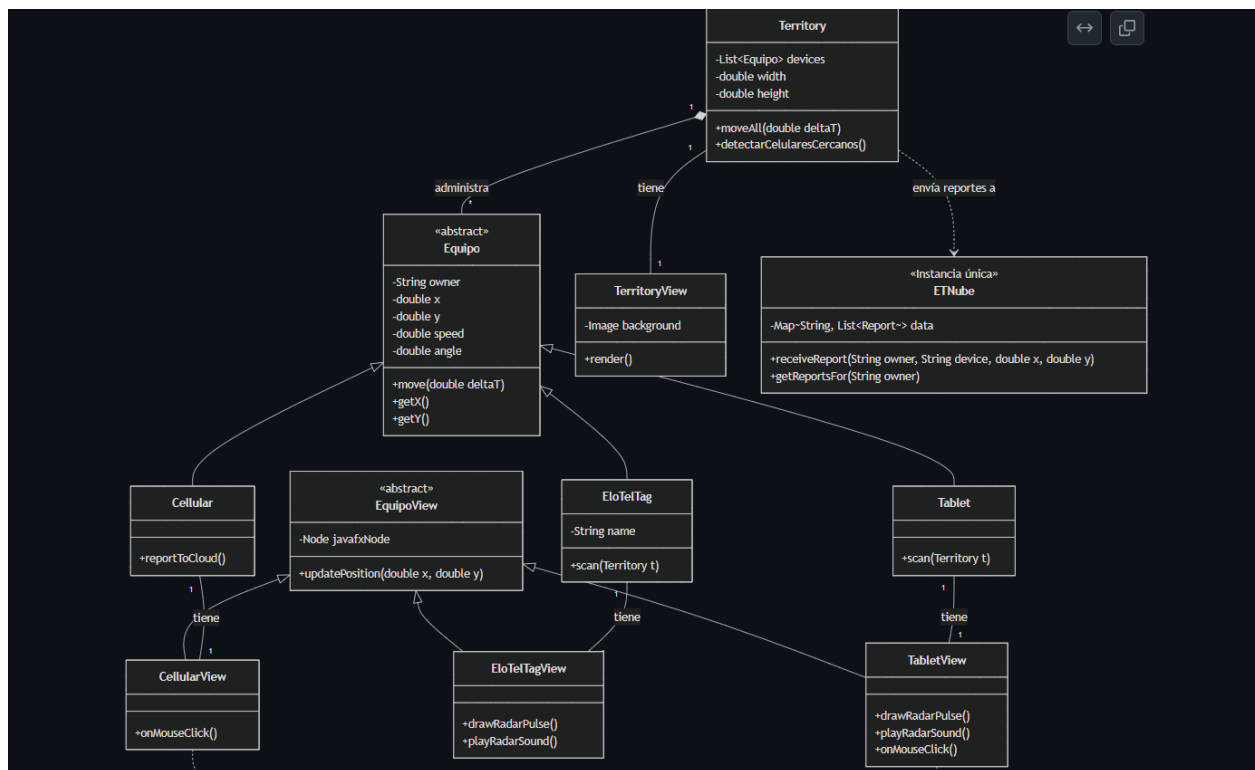


Figura 1: Diagrama de clases parte superior

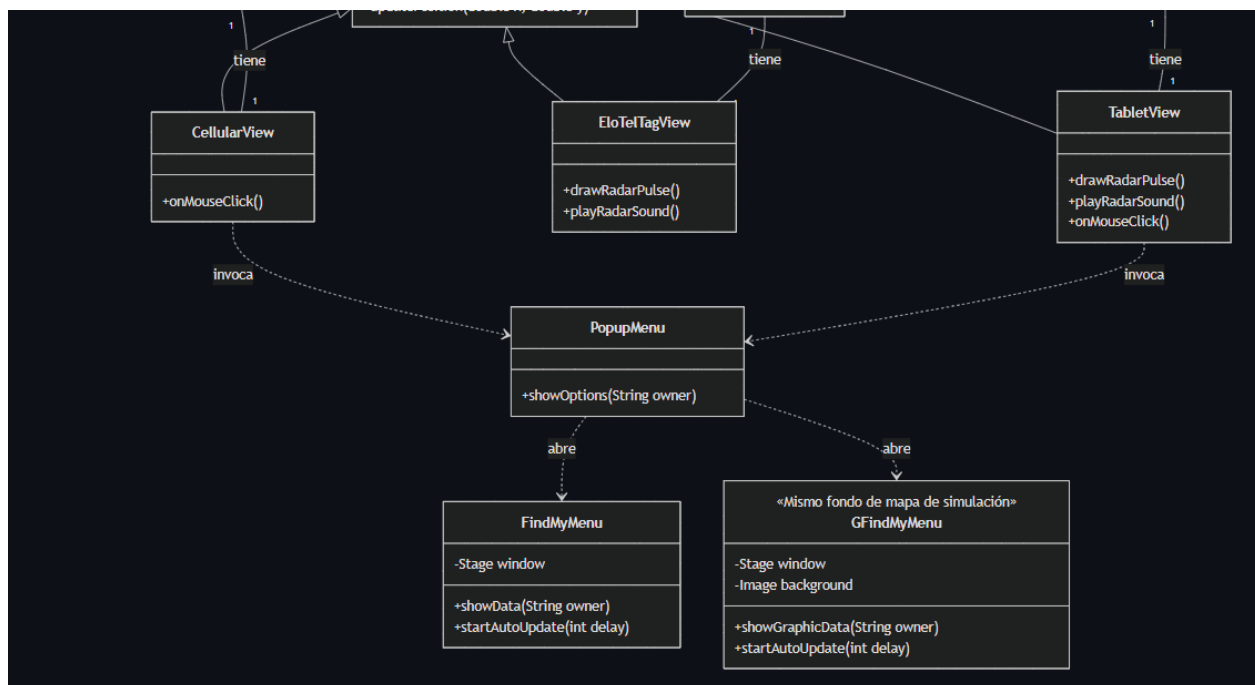


Figura 2: Diagrama de clases parte inferior

3. Dificultades Presentadas y Soluciones Implementadas

3.1. Portabilidad y configuración del entorno JavaFX en distintos dispositivos

Descripción del problema: Al trabajar en un equipo con múltiples integrantes utilizando distintos sistemas operativos (Windows y Linux), la carga manual de las variables de entorno de JavaFX y la declaración estática de las rutas locales de las librerías provocaban fallas críticas de compilación al sincronizar los cambios, interrumpiendo el flujo del repositorio.

Solución implementada: Se migró la gestión de construcción del proyecto hacia **Maven**, centralizando la inyección automatizada de las dependencias nativas de JavaFX en el archivo `pom.xml`. Complementariamente, se estructuró un archivo **Makefile** acoplado al Maven Wrapper (`mvnw`), abstrayendo los comandos por consola para garantizar que la compilación y ejecución sean completamente portables e independientes del entorno local de cada desarrollador.

3.2. Actualización dinámica por tiempos estimados

Descripción del problema: La especificación técnica exige que las ventanas flotantes de rastreo mantengan un refresco automatizado de datos cada un segundo. Al intentar implementar bucles iterativos tradicionales de espera programada mediante bloqueos de hilo (`Thread.sleep()`), la cola de eventos del hilo principal de renderizado (JavaFX Application Thread) se saturaba, congelando por completo la animación de los equipos en el mapa general.

Solución implementada: Se aislaron los ciclos de actualización gráfica encapsulándolos dentro de componentes `Timeline` independientes configurados en modo indefinido. De esta forma, el temporizador secundario realiza consultas no bloqueantes a la instancia de `ETNube` en intervalos exactos de un segundo, redibujando los componentes de texto e íconos en `FindMyMenu` y `GFindMyMenu` sin interferir con la fluidez del mapa central.

3.3. Cálculos de proximidad no detallados correctamente ó con problemas de sincronización

Descripción del problema: Las especificaciones del negocio determinan que los dispositivos de rastreo (Tags y Tablets) emiten pulsos de onda cíclicos para detectar de manera exclusiva teléfonos móviles. Si el motor de colisiones realizaba el cálculo de proximidad de forma genérica sobre la lista global de equipos, las Tablets se detectaban entre sí o los Tags enviaban reportes redundantes, corrompiendo la fidelidad de la base de datos.

Solución implementada: Se implementó una lógica de control de tipos mediante el operador `instanceof` acoplado a la flexibilidad del polimorfismo dentro del método `detectarCelularesCercanos` de la clase `Territory`. Con este filtro, el bucle descarta cualquier choque que no pertenezca estrictamente a la subclase `Cellular` antes de proceder al cálculo de la distancia euclidiana de 50 píxeles, logrando una transferencia limpia de información hacia la nube.